

GABARITO DA LISTA DE EXERCÍCIOS

CONHECENDO A ÁREA DE CONHECIMENTO REQUISITOS DE *SOFTWARE* (*SOFTWARE REQUIREMENTS*) DO SWEBOK

Última atualização: 16/02/2025

Esta lista de exercícios é uma continuação da lista de exercícios anterior sobre a área de conhecimento “Requisitos de software” do SWEBOK.

Exercício 1– Explique como a rastreabilidade de requisitos pode auxiliar no impacto de mudanças em projetos de software.

A rastreabilidade de requisitos é fundamental para entender como mudanças nos requisitos impactam outras áreas do projeto, como *design*, implementação e testes. Quando a rastreabilidade está bem estabelecida, é possível identificar rapidamente quais componentes do sistema serão afetados por uma mudança nos requisitos, ajudando na análise do impacto da alteração.

Por exemplo, se um requisito de segurança for alterado, a rastreabilidade permitirá que a equipe de desenvolvimento identifique quais módulos do sistema e quais testes precisarão ser ajustados. Além disso, a rastreabilidade facilita a comunicação entre as equipes de desenvolvimento, análise e testes, garantindo que todas as mudanças sejam gerenciadas adequadamente e que os impactos sejam mitigados. Isso contribui para o controle de qualidade e a redução de retrabalho, além de manter o projeto alinhado com os objetivos do negócio.

Exercício 2 – Métodos como ATDD e BDD são usados para reduzir ambiguidades nos requisitos. Escolha um deles e explique como funciona e seus benefícios.

BDD (Behavior-Driven Development) é uma técnica de desenvolvimento de *software* que se foca na colaboração entre desenvolvedores, *testers* e *stakeholders* para garantir que o sistema atenda às expectativas dos usuários.

No BDD, os requisitos são descritos usando a sintaxe Gherkin, que permite expressar os comportamentos esperados de forma simples e clara, usando linguagem natural. A sintaxe Gherkin segue a estrutura "Dado, Quando, Então" (Given, When, Then), facilitando a comunicação entre as equipes. Por exemplo, um caso de teste em Gherkin pode ser:

“Dado que o usuário está na página de login, Quando ele insere credenciais válidas, Então ele deve ser redirecionado para a página inicial.”

Esses casos de teste descritos em linguagem natural podem ser usados para gerar testes automatizados em linguagens como Java, que validam se a aplicação se comporta como esperado.

Os benefícios do BDD incluem a redução de ambiguidades nos requisitos, já que as partes interessadas podem entender os testes e os comportamentos esperados de forma clara. Além disso, melhora a colaboração entre as equipes e permite que o sistema seja desenvolvido de acordo com as necessidades do usuário, evitando retrabalho e aumentando a qualidade do software.

Exercício 3 – Quais são os principais desafios da elicitação de requisitos? Cite e explique pelo menos três.

A elicitação de requisitos é uma atividade essencial para o levantamento dos requisitos iniciais de um sistema de *software*. No entanto, é um processo desafiador que envolve reuniões e entrevistas com diversos *stakeholders*, análise de sistemas similares, leitura de documentação e regulamentações pertinentes, entre outras atividades. Esses desafios podem ser ainda mais difíceis quando realizados por pessoas inexperientes.

Alguns dos principais desafios são:

- **Garantir requisitos suficientes:** Um dos maiores desafios é saber se os requisitos levantados são suficientes para atender às necessidades do cliente e permitir o início do desenvolvimento do projeto. Muitas vezes, durante o desenvolvimento, é identificado que alguns requisitos importantes não foram levantados, o que pode levar a mudanças no escopo e atrasos no projeto.
- **Evitar requisitos conflitantes:** Durante a elicitação, é comum que diferentes *stakeholders* apresentem requisitos que entram em conflito entre si. Por exemplo, um cliente pode exigir um sistema muito rápido, enquanto outro pode demandar funcionalidades complexas que impactam no desempenho. Identificar esses conflitos de maneira precoce e gerenciá-los é um grande desafio.
- **Priorizar requisitos:** Nem todos os requisitos têm a mesma importância. Priorizar corretamente quais requisitos devem ser implementados primeiro e quais podem ser adiados é fundamental para o sucesso do projeto. Isso pode ser particularmente difícil quando há muitos *stakeholders* envolvidos e as prioridades de cada um não estão alinhadas.

Exercício 4 – Qual(is) a(s) diferença(s) entre Software e Sistemas?

A diferença entre software e sistemas está relacionada ao escopo, à natureza e à forma como ambos se integram no contexto de tecnologia e computação.

O software é o conjunto de programas, aplicativos ou sistemas operacionais que rodam em um computador ou dispositivo. Ele inclui todos os códigos, algoritmos e rotinas que permitem que um sistema de *hardware* execute funções específicas. O software pode ser aplicativo (como um processador de texto ou jogo), sistema operacional (como o Windows, Linux ou MacOS), firmware (*software* incorporado a dispositivos), ou software de desenvolvimento (como compiladores e editores).

Um sistema é um conjunto de componentes ou entidades interligadas que trabalham juntas para atingir um objetivo comum. Em um contexto de tecnologia, um sistema pode incluir tanto *hardware* (como computadores, servidores, dispositivos móveis) quanto *software*, além de processos, pessoas e até regras de negócio. O termo sistema pode se referir a um sistema computacional, sistema de informação, sistema embarcado, sistema distribuído, ou até sistema de controle, entre outros. Em um contexto mais amplo, sistema engloba não apenas o *software*, mas também o *hardware*, os processos, os usuários, legislação e instituições. Alguns exemplos de sistemas:

Exemplo 1: Um sistema bancário não é apenas o *software* que realiza transações, mas também inclui servidores de banco de dados, interfaces de usuário, regras de segurança e protocolos de comunicação. Assim, o sistema envolve *software*, *hardware*, pessoas (como caixas eletrônicos e bancários) e procedimentos (como autenticação de clientes).

Exemplo 2: Um sistema de controle de temperatura em um reator nuclear inclui sensores (*hardware*), o *software* que analisa os dados dos sensores e ativa mecanismos de resfriamento, além de processos operacionais que indicam como agir em situações críticas.

Exemplo 3: O *software* de um *smartphone* (como o Android) faz parte de um sistema maior, que inclui o próprio hardware (tela, bateria, processador) e os usuários (pessoas que interagem com o *smartphone*). O sistema todo precisa do *software* para operar, mas também depende do hardware e das interações humanas para funcionar.

Assim sendo, o *software* é parte de um sistema, mas um sistema não se limita ao *software*. Um sistema pode existir sem *software*, se considerarmos, por exemplo, um sistema mecânico (como um motor ou máquina), mas o termo sistema computacional sempre envolve *software*. Em suma, o *software* é uma parte vital de um sistema, mas um sistema pode ser muito mais complexo e envolver interações entre diferentes componentes além do *software*.

Exercício 5 – Qual(is) a(s) semelhança(s) entre Software e Sistemas?

Embora *software* e sistemas tenham diferenças claras, também existem várias semelhanças entre eles, especialmente no contexto de tecnologia e computação. Aqui estão as principais semelhanças:

- **Objetivo Comum de Resolução de Problemas:** Tanto *software* quanto sistemas são projetados para resolver problemas e atender a necessidades. O *software* resolve problemas específicos por meio de funções e operações, enquanto o sistema resolve problemas mais amplos, integrando hardware, *software* e processos.
 - Exemplo: Um sistema de gerenciamento de banco de dados (DBMS) é um sistema que inclui o *software* que gerencia o armazenamento e a recuperação de dados.
- **Interdependência entre Componentes:** Ambos envolvem a interação de componentes para alcançar uma finalidade. No caso do *software*, os programas interagem para executar tarefas, e em um sistema, diversos componentes (hardware, *software*, redes, etc.) interagem para cumprir objetivos.
 - Exemplo: Um sistema de controle de temperatura pode ter sensores (hardware), o *software* que interpreta esses dados e aciona o resfriamento, e os usuários que ajustam a configuração. Todos esses elementos precisam interagir para o funcionamento do sistema como um todo.

3. Necessidade de Atualização e Manutenção:

- **Software e sistemas** requerem **atualizações regulares e manutenção contínua** para garantir seu bom funcionamento. Ambos podem ser afetados por erros, falhas e necessidades de aprimoramento ao longo do tempo.
 - Exemplo: O *software* de um computador precisa ser atualizado para corrigir bugs e melhorar a segurança, e o sistema de TI de uma empresa precisa de manutenção para garantir que o hardware, o *software* e os processos funcionem corretamente.

4. Desenvolvimento e Implementação:

- Tanto **software** quanto **sistemas** passam por processos de **desenvolvimento, implementação e testes** para garantir que funcionem conforme o esperado. Um *software* pode ser desenvolvido como parte de um sistema maior, e o desenvolvimento de um sistema geralmente envolve o design e a implementação de *software*.
 - Exemplo: Ao criar um sistema de automação residencial, é necessário desenvolver **software** para controlar os dispositivos e integrar esse *software* ao sistema físico da casa (sensores, câmeras, etc.).

5. Eficiência e Desempenho:

- Tanto o **software** quanto os **sistemas** buscam **eficiência** no uso de recursos e **alto desempenho** para garantir que as tarefas sejam realizadas de forma eficaz e dentro de um prazo aceitável.
 - Exemplo: O software de compressão de dados busca otimizar a quantidade de dados armazenados ou transmitidos, enquanto o sistema de comunicação em rede otimiza a entrega desses dados entre dispositivos.

6. Tecnologia e Inovação:

- Ambos estão **relacionados à tecnologia** e ao **avançar das inovações**. O desenvolvimento de novos softwares pode impulsionar o progresso em diferentes áreas, e novos tipos de sistemas podem abrir portas para novas soluções e aplicativos.
 - Exemplo: O software de inteligência artificial (IA) está cada vez mais integrado a sistemas complexos, como os de veículos autônomos, para otimizar a navegação e a segurança.

Resumo das Semelhanças:

- **Objetivo comum:** Ambos buscam resolver problemas e atender a necessidades.
- **Interdependência:** Componentes de software ou sistemas interagem para alcançar os objetivos.
- **Atualizações e manutenção:** Ambos precisam de manutenção contínua e atualização para manter o desempenho e a segurança.
- **Desenvolvimento e implementação:** Ambos passam por processos de desenvolvimento e teste para garantir que funcionem corretamente.
- **Eficiência e desempenho:** Ambos buscam otimizar o uso de recursos e melhorar a performance.
- **Tecnologia e inovação:** Tanto software quanto sistemas estão fortemente ligados ao avanço tecnológico.

Em essência, embora software e sistemas se refiram a conceitos distintos, eles compartilham o objetivo de resolver problemas e alcançar funcionalidades que atendam aos usuários ou às necessidades de uma organização.

6. A área de Engenharia de Sistemas também usa o conceito de requisitos?

Sim, a área de Engenharia de Sistemas utiliza amplamente o conceito de requisitos. Requisitos são fundamentais para definir **o que** um sistema precisa fazer (**requisitos funcionais**) ou as **condições e restrições** sob as quais ele deve operar (**requisitos não funcionais**). Esses requisitos orientam todo o ciclo de vida do sistema, desde o planejamento, projeto e desenvolvimento até a integração, testes e manutenção.

7. Na área de Engenharia de Sistemas, em nível de negócio/*stakeholders*, o termo “necessidade” (needs) é bastante difundido. Qual a relação entre *Business needs* (necessidades de negócios), *Stakeholder’s needs* (necessidades de *stakeholders*) e *Requirements* (Requisitos)?

De maneira resumida, a principal relação entre esses termos é que os **requisitos surgem da tradução das necessidades (tanto de negócios quanto dos stakeholders) para um conjunto claro e mensurável de funcionalidades e características que o sistema deve possuir para ser considerado bem-sucedido.**

De maneira detalhada, na engenharia de sistemas, os termos *Business Needs* (necessidades de negócios), *Stakeholder’s Needs* (necessidades dos *stakeholders*) e Requisitos estão profundamente interligados, pois

todos dizem respeito a desejos, capacidades, expectativas e objetivos que devem ser atendidos pelo sistema a ser desenvolvido. A seguir, explico a relação entre esses conceitos:

Business Needs (Necessidades de Negócios):

- **Definição:** Refere-se às necessidades e objetivos de uma organização ou negócio. Essas necessidades geralmente são de natureza estratégica e envolvem aspectos como aumento de eficiência, redução de custos, expansão de mercado, melhoria de processos e atendimento a exigências regulatórias.
- **Exemplo:** Uma empresa de transporte público pode ter a necessidade de aumentar a eficiência operacional, reduzir os custos com manutenção ou melhorar a experiência do usuário em seu sistema de transporte.

Stakeholder's Needs (Necessidades dos Stakeholders):

- **Definição:** São as necessidades e expectativas dos diversos *stakeholders* (partes interessadas) de um sistema. *Stakeholders* podem incluir clientes, usuários finais, gerentes, fornecedores, investidores, entre outros.
- **Exemplo:** Em um sistema de saúde, um *stakeholder* como o médico pode ter a necessidade de acesso rápido a registros de pacientes, enquanto um paciente pode precisar de informações claras e rápidas sobre seu tratamento.

Requisitos:

- **Definição:** Requisitos são descrições detalhadas de como um sistema ou produto deve comportar-se ou funcionar para atender às necessidades do negócio e dos *stakeholders*. De maneira simplificada, eles podem ser funcionais (o que o sistema deve fazer) ou não funcionais (como o sistema deve fazer).
- **Exemplo:** Um requisito funcional pode ser:

"O sistema deverá permitir que o médico visualize o histórico completo do paciente em uma interface intuitiva",

- **Exemplo:** enquanto um requisito não funcional (*performance*, neste exemplo) pode ser:

"O sistema deve ser capaz de carregar o histórico do paciente em até 2 segundos".

Relação entre Business Needs, Stakeholder's Needs e Requisitos:

- **Business Needs** são a motivação de alto nível para o desenvolvimento de um sistema. Eles definem os objetivos estratégicos da organização, como aumentar a competitividade, reduzir custos ou melhorar o atendimento ao cliente.
 - **Exemplo:** Uma empresa quer melhorar a satisfação do cliente ao reduzir o tempo de resposta em seu serviço de atendimento.
- **Stakeholder's Needs** surgem da interação com as partes interessadas. Cada *stakeholder* tem necessidades específicas, que podem ser diferentes dependendo de seu papel no sistema. Essas necessidades devem ser capturadas e priorizadas para garantir que o sistema atenda aos diversos interesses dos envolvidos.

- **Exemplo:** O cliente quer que o serviço de atendimento ao cliente seja rápido, enquanto o supervisor deseja uma ferramenta para monitorar a *performance* da equipe.
- **Requisitos** são as transformações dessas necessidades em detalhes específicos sobre o que o sistema precisa fazer e como ele deve se comportar para atender às expectativas tanto do negócio quanto dos *stakeholders*.
 - **Exemplo:** Para atender à **necessidade do cliente de respostas rápidas**, um requisito pode ser:

"o sistema de atendimento deverá ser capaz de responder a uma consulta de cliente em até 30 segundos".

Processo de Tradução das Necessidades em Requisitos:

- **Identificação das Necessidades:** O processo começa com a identificação das necessidades de negócio e as necessidades dos *stakeholders*. Essas necessidades podem ser obtidas por meio de reuniões, entrevistas, pesquisa de mercado, análise de concorrência, etc.
- **Análise e Priorização:** Uma vez identificadas as necessidades, elas são analisadas e priorizadas, levando em consideração a viabilidade, o impacto no negócio, os recursos disponíveis e os limites técnicos.
- **Especificação de Requisitos:** As necessidades são então convertidas em requisitos técnicos, detalhando funcionalidades e qualidades do sistema, como ele deve se comportar, o desempenho esperado e os padrões de segurança e usabilidade a serem atendidos.

Exemplo de Processo de Tradução:

1. **Business Need:** A empresa deseja aumentar a satisfação do cliente e reduzir o tempo de espera no atendimento.
2. **Stakeholder's Need:** O gerente de atendimento deseja uma ferramenta que permita identificar rapidamente a demanda dos clientes e priorizar os atendimentos.
3. **Requisitos:**
 - **Funcional:** "O sistema de atendimento deve classificar automaticamente as consultas de clientes com base na urgência."
 - **Não Funcional:** "O sistema deve ser capaz de processar 1.000 consultas simultaneamente, sem afetar o desempenho."

Resumo da Relação:

- **Business Needs** definem os objetivos estratégicos e motivações da organização.
- **Stakeholder's Needs** refletem as necessidades específicas de todos os envolvidos no sistema.
- **Requisitos** são as especificações detalhadas de como o sistema atenderá essas necessidades.